

Proyecto Programado Estructuras Dinámicas Lineales- Clash Royale

Estructura de Datos

I-Semestre 2026

Profesora:

Raquel Fabiana Mora Rojas

Estudiantes:

Fabricio Alejandro Castro Muñoz - 2025120450

Josué David González Alvarado - 2024297026

Mariana González Céspedes - 2025077428

Índice

Descripción del problema.....	2
Justificación.....	3
Solución propuesta.....	4
Análisis de resultados.....	10
Conclusiones.....	14
Bibliografía y Herramientas.....	15

Descripción del problema

El presente proyecto tomó como dominio de aplicación el videojuego *Clash Royale*, un juego de estrategia en tiempo real ampliamente conocido que presenta una arquitectura de datos lo suficientemente compleja como para justificar el uso de múltiples estructuras de datos interconectadas.

El problema central que se busca resolver es la gestión computacional eficiente de seis entidades fundamentales del universo del juego: Cartas, Jugadores, Mazos, Clanes, Arenas y Batallas. Cada una de estas entidades no existe de forma aislada; por el contrario, se relacionan entre sí de maneras múltiples y jerárquicas. Por ejemplo, un jugador pertenece a exactamente una arena determinada por su cantidad de trofeos, puede integrarse a un clan, posee uno o varios mazos, y cada mazo contiene una colección específica de exactamente ocho cartas. Las batallas, a su vez, enfrentan a dos jugadores con sus respectivos mazos dentro de una arena, y deben registrar todos los resultados del enfrentamiento.

La complejidad técnica del sistema se intensifica por tres restricciones fundamentales impuestas en el enunciado del proyecto:

No se permite el uso de las estructuras contenedoras de la biblioteca estándar de C++. Esto obliga a construir desde cero cada estructura de datos, gestionando directamente la memoria dinámica y controlando manualmente los punteros que interconectan los nodos.

Cada entidad no solo debe almacenarse en una lista enlazada, sino que el tipo de lista varía según la entidad: lista simple, lista doble, lista circular, o combinaciones de estas.

El sistema debe modelar las relaciones entre entidades sin duplicar la información. Es decir, si un jugador pertenece a un clan, el clan no almacena una copia del jugador, sino un enlace (puntero o referencia por ID) hacia el nodo original del jugador en su lista principal.

Entre los desafíos técnicos más relevantes se pueden identificar los siguientes: la validación de que un mazo contenga exactamente ocho cartas sin repetición antes de ser confirmado; la verificación de que los trofeos de un jugador correspondan al rango de la arena que se le asigna; la comprobación de que los mazos utilizados en una batalla pertenezcan efectivamente a los jugadores participantes; y la garantía de unicidad de todos los identificadores del sistema.

Justificación

La implementación de un sistema de esta magnitud permite poner en práctica simultáneamente los cuatro tipos de listas enlazadas estudiadas: simple, doble, circular y doble circular, así como el concepto de sublistas enlazadas como mecanismo para representar relaciones de pertenencia.

Desde el punto de vista técnico, el uso de listas enlazadas dinámicas ofrece ventajas fundamentales frente a los arreglos estáticos. En primer lugar, la flexibilidad de tamaño: mientras que un arreglo debe declararse con un tamaño fijo en tiempo de compilación, una lista enlazada puede crecer o reducirse según las necesidades del sistema en tiempo de ejecución, sin desperdiciar memoria ni imponer límites artificiales. En segundo lugar, la eficiencia en inserción y eliminación: en una lista enlazada, insertar o eliminar un nodo en cualquier posición tiene un costo de $O(1)$ una vez que se tiene la referencia al nodo previo, mientras que en un arreglo estas operaciones requieren desplazamiento de elementos. (Stanojević, 2021)

Más allá de las ventajas técnicas generales, el proyecto se justifica concretamente en tres dimensiones:

- **Integridad de datos:** Las validaciones estrictas implementadas (unicidad de IDs, rangos válidos de coronas y duración, costos de elixir entre 1 y 9, mazos con exactamente ocho cartas sin repetición, correspondencia entre trofeos y arena) garantizan que el sistema nunca almacene información incoherente. Esto es fundamental en cualquier sistema de información real, donde los datos incorrectos pueden propagarse y generar errores en cadena.
- **Modelado fiel de relaciones:** El uso de sublistas enlazadas dentro de los nodos de otras listas (como la sublista de cartas dentro de un mazo, o la sublista de jugadores dentro de un clan) permite representar con precisión las relaciones de pertenencia que existen en el universo del juego. Esto demuestra que las estructuras de datos no son herramientas aisladas, sino componentes que se combinan para modelar realidades complejas.
- **Modularidad del diseño:** Al encapsular cada operación en funciones independientes (buscar, insertar, eliminar, actualizar, consultar, reportar) y organizar el código por entidad, se logra un sistema cuyas partes pueden probarse, corregirse y mejorarse de forma independiente, reduciendo el impacto de los errores y facilitando el trabajo en equipo.

Solución propuesta

Para dar solución al problema planteado, se diseñaron e implementaron seis estructuras de datos principales, cada una representada como un struct en C++ con sus respectivos atributos y punteros de enlace. La selección del tipo de lista para cada entidad se fundamentó en las propiedades formales de cada variante, adaptadas al dominio específico del problema (Weiss, 2013).

Cartas(Lista simple con inserción al inicio):

La estructura Cartas almacena la información de cada carta disponible en el sistema. Se implementó como una lista simplemente enlazada con inserción al inicio, lo cual garantiza $O(1)$ para cada nueva inserción. Cada nodo contiene los campos IDCarta (entero único identificador), nombre (cadena de texto), rareza (Común, Rara, Épica o Legendaria), tipo (Tropa, Edificio o Hechizo), costoElixir (entero entre 1 y 9), dañoBase (valor real de daño) y vidaBase (puntos de vida enteros). El puntero sig enlaza cada nodo con el siguiente de la lista.

Validaciones asociadas: No se permiten IDs repetidos, costos de elixir fuera del rango 1-9, ni valores negativos para daño o vida.

Jugadores(Lista doble):

La estructura Jugadores se implementó como una lista doblemente enlazada. Cada nodo contiene dos punteros (sig y ant) que permiten recorrer la lista en ambas direcciones, lo cual es fundamental para mantener el orden durante las inserciones y para generar reportes ordenados eficientemente. Los atributos del jugador incluyen IDJugador, nombreUsuario, nivelRey (nivel del rey de la torre, entre 1 y 14), trofeos, IDArena e IDClan.

El ordenamiento alfabético se mantiene en tiempo de inserción: cada nuevo jugador se ubica en la posición correcta recorriendo la lista y comparando nombres, lo que tiene un costo promedio de $O(n)$. Esto elimina la necesidad de ordenar la lista posteriormente para los reportes.

Validaciones asociadas: No se permiten IDs ni nombres de usuario duplicados. Los trofeos del jugador deben estar dentro del rango (trofeosMin, trofeosMax) de la arena asignada. El IDArena e IDClan deben corresponder a entidades existentes en sus respectivas listas.

Mazos(Lista simple con inserción al final y sublista de cartas):

La estructura Mazos se implementó como una lista simplemente enlazada con inserción al final. Cada mazo pertenece a un único jugador (referenciado por IDJugador) y contiene un campo tipoMazo que clasifica la estrategia del mazo. El atributo más importante desde el punto de vista estructural es listaCartas, que es una sublista simplemente enlazada de nodos NodoCartaMazo. Esta sublista almacena los IDs de las ocho cartas que componen el mazo, sin duplicar la información de las cartas, sino apuntando a ellas por referencia de ID.

El campo cantidadCartas se mantiene actualizado con cada inserción y se usa para validar que el mazo tenga exactamente ocho cartas antes de ser confirmado.

Validaciones asociadas: Un mazo no puede confirmarse con más ni menos de ocho cartas. No se pueden repetir cartas dentro del mismo mazo. El IDJugador debe corresponder a un jugador existente. Todos los IDCarta de la sublista deben existir en la lista de cartas.

Clanes (Lista circular con inserción al final y sublista de jugadores):

La estructura Clanes se implementó como una lista circularmente enlazada con inserción al final. En una lista circular, el último nodo apunta de regreso al primero, lo que permite recorrer la lista de manera continua sin un "fin" explícito. Esta decisión de diseño refleja conceptualmente la naturaleza de los clanes en el juego: son grupos cerrados y continuos. Los atributos incluyen IDClan, nombreClan, region, cantidadMiembros y puntajeClan.

Al igual que los mazos, cada clan contiene una sublista llamada listaJugadores, compuesta por nodos NodoJugadorClan que almacenan el IDJugador de cada miembro. Cuando un jugador se agrega a un clan, se actualiza automáticamente el campo cantidadMiembros del clan.

Validaciones asociadas: Un jugador no puede pertenecer a más de un clan simultáneamente. No se pueden registrar clanes con IDs o nombres duplicados.

Arenas (Lista simple con inserción al inicio):

La estructura Arenas se implementó como una lista simplemente enlazada con inserción al inicio. Cada arena define un rango de trofeos (trofeosMin y trofeosMax) que determina a qué jugadores pertenece. Los atributos son IDArena, nombreArena, trofeosMin y trofeosMax.

Aunque su estructura es la más simple del sistema, las arenas son una entidad central porque actúan como clasificador de jugadores y escenario de batallas. Su correcta implementación es indispensable para las validaciones de trofeos de jugadores y para las consultas que preguntan cuántos jugadores hay en cada arena.

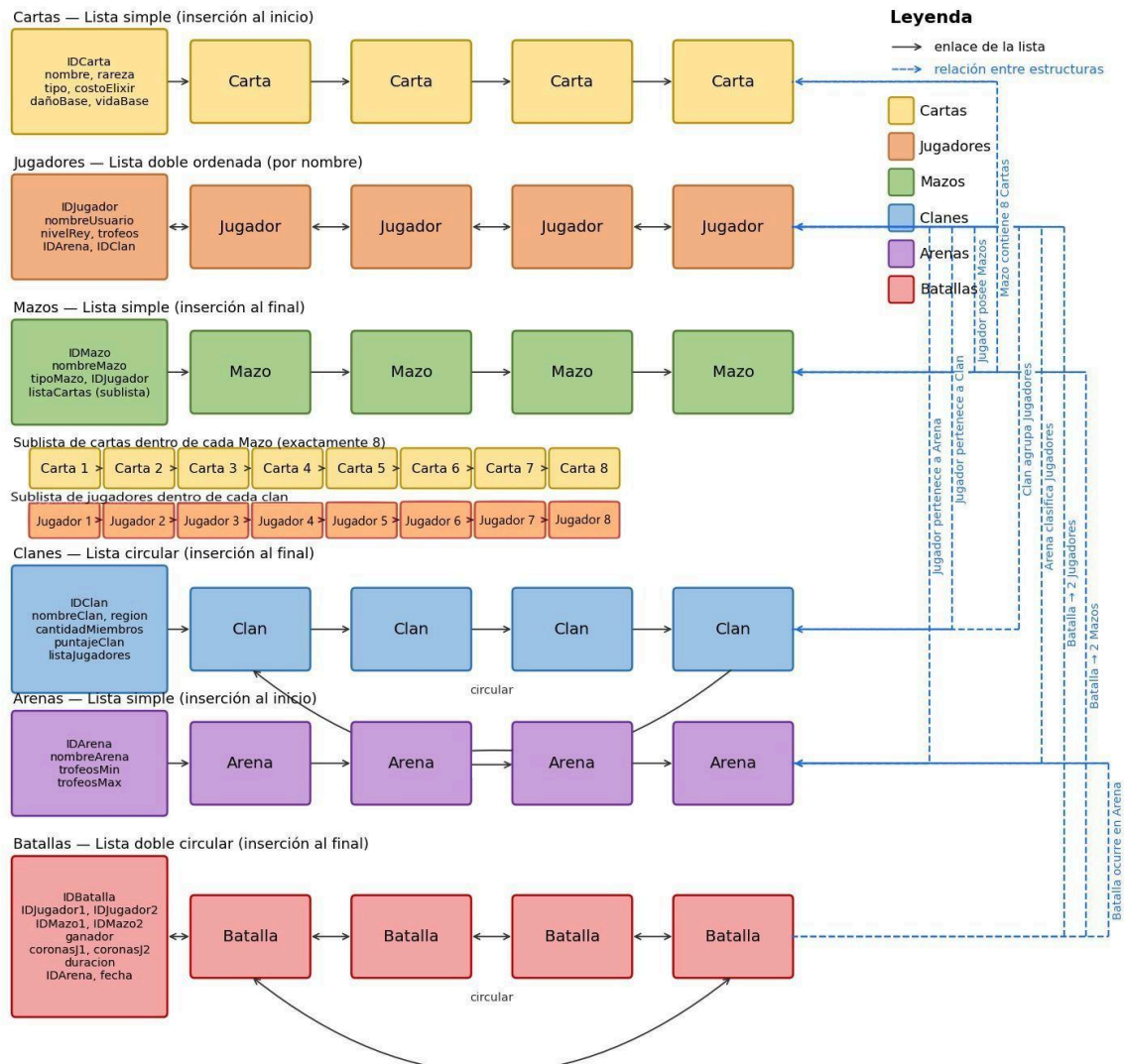
Validaciones asociadas: No se permiten IDs ni nombres de arena duplicados. Los rangos de trofeos no deben solaparse entre arenas distintas.

Batallas (Lista doble circular con inserción al final):

La estructura Batallas es la más compleja del sistema desde el punto de vista de sus relaciones. Se implementó como una lista doblemente enlazada circular con inserción al final. Esto permite recorrer las batallas en ambas direcciones y de forma continua (útil para simulaciones). Cada nodo almacena IDBatalla, IDJugador1, IDJugador2, IDMazo1, IDMazo2, ganador (nombre del jugador ganador o "Empate"), coronasJ1, coronasJ2 (entre 0 y 3 cada una), duracion (valor positivo), IDArena y fecha.

Validaciones asociadas: Ambos jugadores deben existir en la lista de jugadores. Los mazos deben existir y pertenecer a sus respectivos jugadores. Las coronas deben estar en el rango [0, 3]. La duración debe ser un valor positivo. No se puede repetir el mismo IDBatalla.

Diagrama:



El sistema fue implementado siguiendo una arquitectura modular organizada en capas de responsabilidad:

Capa de definición de estructuras: Todos los struct se declaran en el área global del archivo fuente, junto con los punteros cabeza de cada lista principal (primerCarta, primerJugador, primerMazo, primerClan, primerArena, primerBatalla), inicializados en NULL.

Capa de búsqueda: Para cada entidad se implementaron funciones de búsqueda por ID y/o por nombre que recorren la lista correspondiente y retornan un puntero al nodo encontrado, o

NULL si no existe. Estas funciones son la base de las validaciones, ya que permiten verificar existencia y unicidad.

Capa de inserción: Cada estructura tiene su función de inserción que: primero valida los datos de entrada, luego crea el nuevo nodo con new, y finalmente lo enlaza correctamente en la lista según su tipo (al inicio, al final, en orden, etc.).

Capa de actualización y eliminación: Se implementaron funciones para modificar los atributos de cada entidad y para eliminar nodos, con el cuidado de reconectar correctamente los punteros vecinos antes de liberar la memoria con delete.

Capa de validación: Las validaciones están encapsuladas dentro de las funciones de inserción y actualización, y reportan mensajes de error al usuario antes de rechazar cualquier dato inválido.

Capa de consultas y reportes: Funciones especializadas que recorren las listas (y sublistas cuando es necesario) para responder preguntas específicas del sistema y generar listados detallados.

Capa de interfaz: Un sistema de menús anidados permite al usuario navegar entre las operaciones disponibles de manera intuitiva.

Carga inicial de datos: La función cargarDatos() se ejecuta al inicio del programa e inserta al menos 10 registros por entidad principal mediante datos quemados en código, garantizando que el sistema tenga información suficiente para probar todas las funcionalidades desde el primer momento.

Minuta de Trabajo:

Link del GitHub: <https://github.com/Mari-222/Clash-Royale.git>

Fecha	Integrante	Actividad
24 abr 2026	billo08	Subida de archivos iniciales del proyecto (todas las estructuras hasta eliminación de cada uno)
24 abr 2026	Josue507Julio	Función inserción de Cartas y Arena(modificación)
24 abr 2026	Josue507Julio	Funciones de inserción terminadas(modificación)
24 abr 2026	Josue507Julio	Actualización de estructuras del proyecto
24 abr 2026	billo08	Subida de archivos
24 abr 2026	Josue507Julio	Funciones de inserción en Arena y Cartas (modificación)
24 abr 2026	Josue507Julio	Funciones de inserción terminadas (versión final)
24 abr 2026	Josue507Julio	Merge con rama principal
25 abr 2026	billo08	Subida de archivos del proyecto
28 abr 2026	Mari-222	Actualización del README
28 abr 2026	Mari-222	Actualización del archivo principal
28 abr 2026	Mari-222	Actualización del archivo principal
30 abr 2026	Josue507Julio	Menú general y submenús de mantenimiento
30 abr 2026	Josue507Julio	Simulación de batalla completa
30 abr 2026	Josue507Julio	Parte opcional de simulación (estadísticas de mazo)
1 mayo 2026	Mari-222	Corrección en modificar jugador

1 mayo 2026	Mari-222	Cambio en insertarJugadorEnClan
1 mayo 2026	Mari-222	Pequeñas correcciones generales
1 mayo 2026	Mari-222	Consultas implementadas
1 mayo 2026	Mari-222	Actualización del archivo principal
1 mayo 2026	Mari-222	Consultas y reportes finalizados
1 mayo 2026	Mari-222	Cambio en menú de consultas
1 mayo 2026	Mari-222	Actualización del archivo principal
1 mayo 2026	Mari-222	Actualización del archivo principal
1 mayo 2026	Mari-222	Validación de elixir
1 mayo 2026	Mari-222	Actualización del archivo principal
1 mayo 2026	billo08	Actualización del archivo principal
1 mayo 2026	billo08	Actualización del archivo principal
2 mayo 2026	Mari-222	Corrección en insertarBatalla
2 mayo 2026	billo08	Actualización del archivo principal
3 mayo 2026	Josue507Julio	Simulación finalizada
5 mayo 2026	billo08	Actualización del archivo principal
5 mayo 2026	billo08	Actualización del archivo principal
7 mayo 2026	billo08	Corrección de modificarCarta

Análisis de resultados

Requerimiento	Estado	Observaciones
Estructuras		
Cartas (lista simple)	Completo	
Jugadores (lista doble ordenada)	Completo	
Mazos (lista simple)	Completo	
Clanes (lista circular)	Completo	
Arenas (lista simple)	Completo	
Batallas (lista doble circular)	Completo	
Sublistas		
Cartas en Mazo	Completo	
Jugadores en Clan	Completo	
Inserciones		
Cartas	Completo	
Jugadores	Completo	
Mazos	Completo	
Clanes	Completo	
Arenas	Completo	
Batallas	Completo	
Carta en Mazo	Completo	
Jugador en Clan	Completo	
Modificaciones		
Cartas	Completo	

Jugadores	Completo	Incluye reordenamiento alfabético y cambio de clan
Mazos	Completo	
Clanes	Completo	
Arenas	Completo	
Batallas	No realizado	Decisión intencional, una batalla es un registro histórico no debería de modificarse
Eliminaciones		
Cartas	Completo	Valida que no esté en uso en algún mazo
Jugadores	Completo	Elimina en cascada sus mazos y lo quita del clan
Mazos	Completo	Libera sublista de cartas
Clanes	Completo	Libera sublista de jugadores
Arenas	Completo	
Batallas	Completo	
Carta de Mazo	Completo	
Jugador de Clan	Completo	
Validaciones		
Unicidad de IDs	Completo	
Rareza y tipo de carta válidos	Completo	
Costo de elixir entre 1 y 9	Completo	
Trofeos compatibles con arena	Completo	
Mazos con exactamente 8 cartas	Completo	
Sin cartas repetidas en mazo	Completo	
Mazos pertenecen al jugador correcto	Completo	
Coronas entre 0 y 3	Completo	

Jugador en un solo clan	Completo	
Solapamiento de rangos entre arenas	Completo	
Carga automática de datos		
10 Cartas	Completo	
10 Jugadores	Completo	
10 Mazos	Completo	
10 Clanes	Completo	
10 Arenas	Completo	
10 Batallas	Completo	Estadísticas de simulación en cero por ser datos históricos
Consultas		
Carta más utilizada en mazos	Completo	
Jugador con más trofeos	Completo	
Clan con más miembros	Completo	
Mazo con menor costo promedio de elixir	Completo	
Arena con más jugadores	Completo	
Jugador con más victorias	Completo	
Batallas por arena	Completo	
Jugadores de un clan	Completo	
Cartas de rareza legendaria	Completo	
Mazos de un jugador	Completo	
Reportes		
Todas las listas principales	Completo	
Detalle de jugadores con arena, clan y mazos	Completo	

Mazos con sus 8 cartas	Completo	
Clanes con sus miembros	Completo	
Arenas con sus jugadores	Completo	
Batallas con detalle	Completo	
Cartas ordenadas por elixir	Completo	Ordenamiento por intercambio de datos entre nodos
Jugadores en orden alfabético	Completo	La lista se mantiene ordenada
Victorias por jugador	Completo	
Simulación de batallas		
Registro de jugadores involucrados	Completo	
Registro de mazos usados	Completo	
Registro de arena	Completo	
Registro de coronas	Completo	
Registro de ganador	Completo	
Registro de duración	Completo	
Promedio de elixir por mazo	Completo	
Tipo predominante de cartas	Completo	
Consumo estimado de elixir	Completo	
Cartas más usadas en simulaciones	Completo	
Interfaz		
Menú principal	Completo	
Submenú mantenimiento	Completo	
Submenú consultas	Completo	
Submenú reportes	Completo	
Submenú simulación	Completo	

Conclusiones

La implementación desde cero de listas enlazadas simples, dobles, circulares y doble circulares en C++, sin apoyo de la STL, exigió un entendimiento profundo del comportamiento de los punteros y la gestión de memoria dinámica. Esta experiencia demostró que las estructuras de la STL, aunque convenientes, abstraen una complejidad que todo programador profesional debe ser capaz de manejar directamente cuando las circunstancias lo requieren, como en sistemas embebidos, entornos con recursos limitados o contextos donde el control fino sobre la memoria es crítico.

El modelado de las relaciones entre entidades mediante punteros y sublistas, en lugar de duplicar datos, evidenció la importancia del principio de integridad referencial en el diseño de sistemas de información. Una modificación en la información de una carta se refleja automáticamente en todos los mazos que la contienen, sin necesidad de actualizar múltiples copias, lo que reduce errores y mejora la coherencia del sistema.

Las validaciones implementadas a nivel de inserción y actualización demostraron ser fundamentales para la estabilidad del sistema. Un sistema sin validaciones puede almacenar datos incoherentes que provocan comportamientos inesperados en operaciones posteriores, como intentar acceder a un mazo que pertenece a un jugador inexistente. La validación temprana de datos es siempre preferible a la corrección tardía de errores.

La elección del tipo de lista para cada entidad no fue arbitraria, sino que respondió a las características operacionales de cada entidad. Una lista circular para los clanes permite recorrerlos de manera continua sin comprobar por un "fin" de lista, lo cual resulta natural en estructuras de grupos. Una lista doble para los jugadores permite mantener el orden alfabético con menor costo que una lista simple, ya que facilita tanto las inserciones en posición correcta como los recorridos bidireccionales. Este proceso de selección de estructuras es uno de los aprendizajes más valiosos del proyecto.

Bibliografía y Herramientas

Weiss, M. A. (2013). *Data Structures and Algorithm Analysis in C++* (4.^a ed.). Pearson Education.

Graph Ltd. (2024). *draw.io* (Versión 24.2.5) [Software de computadora]. diagrams.net

Stanojević, (2021). *Estructuras de datos fundamentales: Listas enlazadas*.
<https://www.vegaitglobal.com/media-center/knowledge-base/fundamental-data-structures-linked-lists>

Microsoft. (2025). *Visual Studio Code* (Versión 1.118.1). <https://code.visualstudio.com>